

RetroSpectrum Security Pipeline and Cryptographic Reference

August 13, 2026

Abstract

This document summarizes the active security pipeline implemented by the supplied RetroSpectrum security sources: `ServerIdentity.c`, `SecureNetwork.c`, `AuthScreen.c`, `AuthAdmin.c`, and `SecureFunctions.c`. It focuses on trust establishment, post-quantum server authentication, TLS transport, password and two-factor authentication, authorization, rate limiting, and defensive file/memory handling. It is intentionally compact and describes the implementation shown in the supplied source rather than a generic security architecture.

Contents

1	Scope and Security Components	2
2	End-to-End Security Pipeline	2
3	Persistent Server Identity and Trust	3
3.1	ML-DSA-87 identity key	3
3.2	Signed discovery announcements	3
4	TLS Transport and Channel-Bound Server Authentication	3
4.1	TLS configuration	3
4.2	ML-DSA proof bound to the TLS session	4
5	User Authentication and Two-Factor Authentication	4
5.1	Password hashing with Argon2id	4
5.2	TOTP secret protection and verification	4
6	Authorization, Session Gating, and Abuse Controls	5
6.1	Role model	5
6.2	Pre-authentication and rate limits	5
7	Defensive Memory and File Handling	6
8	Cryptographic Primitive Summary	6
9	Compact Security Reference	7

1 Scope and Security Components

The security path is divided across five principal modules:

Module	Primary security role
<code>ServerIdentity.c</code>	Persistent ML-DSA-87 server identity, SHA-512 fingerprinting, trusted <code>.rspub</code> public-key files, and signed LAN identity announcements.
<code>SecureNetwork.c</code>	TLS 1.3 transport, hybrid key exchange, channel-bound ML-DSA identity proof, framing, pre-authentication limits, session state, and remote operation dispatch.
<code>AuthScreen.c</code>	Argon2id password storage, HMAC-SHA512 TOTP, AES-256-GCM protection of TOTP secrets, login rate limiting, TOTP replay prevention, and local/remote authentication flow.
<code>AuthAdmin.c</code>	Administrator/co-administrator account-management policy and user-interface enforcement.
<code>SecureFunctions.c</code>	Bounded string/memory operations, checked allocation, validated file handling, private-directory checks, and <code>O_NOFOLLOW</code> -based path hardening.

The authentication database is opened through `DATABASE_CRYPT0_open_auth()`, but the implementation of `DatabaseCrypto.c` was not among the supplied files. Therefore this reference does not assign a specific database-at-rest cipher to that layer.

2 End-to-End Security Pipeline

For a remote client, the active security path is:

```

Trusted .rspub public key selected/imported
↓
TCP connection to the trusted host
↓
TLS 1.3:  SecP384r1MLKEM1024 + TLS_AES_256_GCM_SHA384
↓
TLS exporter + client nonce + server nonce
↓
ML-DSA-87 signature verifies the trusted server on this TLS channel
↓
Encrypted AUTH frame:  username + password + optional TOTP
↓
Argon2id password verification
↓
If enabled:  decrypt TOTP secret, verify HMAC-SHA512 TOTP, reject reused counter
↓
Apply role policy and create authenticated session
↓
Authenticated SAVE / LOAD / LIST / DELETE operations; admin operations require
privilege

```

The important separation is that TLS first provides an encrypted channel, then the imported ML-DSA-87 identity authenticates that exact channel before application credentials are accepted. Only after user authentication does the server permit normal application requests.

3 Persistent Server Identity and Trust

3.1 ML-DSA-87 identity key

The server generates a persistent ML-DSA-87 key pair through OpenSSL 3.5+.

Relevant code (ServerIdentity.c lines 437–462):

```
437 static int server_identity_generate_keypair(...)
449     key = EVP_PKEY_Q_keygen(NULL, NULL, SERVER_IDENTITY_ALGORITHM);
451     if (!key || EVP_PKEY_get_raw_private_key(key, private_key, &private_size) != 1 ||
452         EVP_PKEY_get_raw_public_key(key, public_key, &public_size) != 1 ||
453         private_size != SERVER_IDENTITY_PRIVATE_KEY_BYTES ||
454         public_size != SERVER_IDENTITY_PUBLIC_KEY_BYTES) {
455         goto cleanup;
457     }
```

The public key is hashed with domain-separated SHA-512 to form the displayed fingerprint. Public identity files contain the server name, ML-DSA-87 public key, and a SHA-512 digest checked with `CRYPTO_memcmp()`. Importing a valid `.rspub` file makes that public key the client's trusted server identity.

The SHA-512 digest protects the format/integrity of the public identity file; the trust decision is the explicit selection/import of that public key. The ML-DSA private-key file is stored in the user's private configuration area and protected by file permissions and a SHA-512 integrity digest in this module; it is not wrapped by an additional private-key encryption layer here.

3.2 Signed discovery announcements

LAN identity packets contain the server ID, public key, a 32-byte random nonce, and a timestamp. The signed portion is authenticated with ML-DSA-87, and received packets must also pass a clock-skew check.

Relevant code (ServerIdentity.c lines 1329–1343):

```
1329 memset(packet, 0, SERVER_IDENTITY_PACKET_BYTES);
1333 memcpy(packet + 10, Global_Server_Identity_Id, SERVER_IDENTITY_ID_LENGTH);
1334 memcpy(packet + SERVER_IDENTITY_PUBLIC_OFFSET, Global_Server_Identity_Public,
1335         SERVER_IDENTITY_PUBLIC_KEY_BYTES);
1336 if (RAND_bytes(packet + SERVER_IDENTITY_NONCE_OFFSET, SERVER_IDENTITY_NONCE_BYTES) != 1)
1338     return 0;
1341 server_identity_write_u64(packet + SERVER_IDENTITY_TIMESTAMP_OFFSET, (uint64_t)time(NULL));
1342 return server_identity_sign_with_private(Global_Server_Identity_Private, packet,
1343                                         SERVER_IDENTITY_SIGNED_BYTES, ...);
```

4 TLS Transport and Channel-Bound Server Authentication

4.1 TLS configuration

RetroSpectrum restricts transport to TLS 1.3 and requires a specific hybrid TLS group and cipher suite.

Relevant code (SecureNetwork.c lines 393–405):

```
393 SSL_CTX_set_min_proto_version(context, TLS1_3_VERSION)
394 SSL_CTX_set_max_proto_version(context, TLS1_3_VERSION)
395 SSL_CTX_set1_groups_list(context, "SecP384r1MLKEM1024")
396 SSL_CTX_set_ciphersuites(context, "TLS_AES_256_GCM_SHA384")
402 SSL_CTX_set_security_level(context, 4);
403 SSL_CTX_set_options(context, SSL_OP_NO_COMPRESSION | SSL_OP_NO_RENEGOTIATION |
404                             SSL_OP_NO_TICKET);
405 SSL_CTX_set_session_cache_mode(context, SSL_SESS_CACHE_OFF);
```

The server creates a temporary secp384r1 EC key and one-day self-signed X.509 certificate. The client deliberately does not trust that ephemeral certificate as the server identity. Instead, after the TLS handshake it requires the imported ML-DSA-87 key to authenticate a channel-bound proof.

4.2 ML-DSA proof bound to the TLS session

The client sends a fresh 32-byte nonce. The server generates another 32-byte nonce. Both sides export 64 bytes of TLS keying material using the label `EXPORTER-RetroSpectrum-PQ-Identity-v1`. The signed proof is conceptually

$$M = \text{domain} \parallel \text{version} \parallel N_C \parallel N_S \parallel E_{TLS}, \quad (1)$$

where N_C and N_S are the client/server nonces and E_{TLS} is the TLS exporter output. The server signs M using the persistent ML-DSA-87 private key; the client verifies it using the trusted imported public key.

This binds the long-term RetroSpectrum server identity to the particular TLS connection rather than relying on the ephemeral X.509 certificate as the trust anchor.

5 User Authentication and Two-Factor Authentication

5.1 Password hashing with Argon2id

Passwords are stored as Argon2id encoded hashes with a fresh random 16-byte salt. The configured parameters are:

Time cost	3
Memory cost	65,536 KiB (64 MiB)
Parallelism	4
Salt	16 bytes
Hash output	32 bytes

Relevant code (AuthScreen.c lines 2470–2500):

```

2470 static int auth_hash_password_argon2id(const char *password, char encoded[...] ) {
2477     if (!password || !encoded || RAND_bytes(salt, sizeof(salt)) != 1)
2478         return 0;
2496     result = argon2id_hash_encoded(AUTH_ARGON2_TIME_COST, AUTH_ARGON2_MEMORY_KIB,
2497                                   AUTH_ARGON2_PARALLELISM, password, strlen(password),
2498                                   salt, sizeof(salt), AUTH_ARGON2_HASH_BYTES, encoded,
2499                                   AUTH_ARGON2_ENCODED_MAX);

```

For nonexistent accounts the server still performs dummy Argon2id work, reducing obvious timing differences between an unknown username and a wrong password.

5.2 TOTP secret protection and verification

When TOTP is enabled, RetroSpectrum uses a 32-byte secret and HMAC-SHA512. The period is 30 seconds and the output is six decimal digits. Verification accepts the current counter plus one adjacent counter on either side.

The stored TOTP secret is encrypted with AES-256-GCM using a random 12-byte nonce and a 16-byte authentication tag. A per-user 256-bit wrapping key is derived from a 32-byte server master key and a random 16-byte per-user salt using two HMAC-SHA512 stages:

$$PRK = \text{HMAC}_{SHA512}(\text{salt}, \text{master-key}), \quad (2)$$

$$K_u = \text{first}_{32}(\text{HMAC}_{SHA512}(PRK, \text{domain} \parallel \text{username} \parallel 01)). \quad (3)$$

The server master key is created with `RAND_bytes()` and stored as an owner-only file.

For login, the secret is decrypted, the HMAC-SHA512 TOTP is checked, and the matching time counter must be greater than the stored `last_totp_counter`. The database update is conditional, preventing the same accepted TOTP time-step from being reused.

Relevant code (AuthScreen.c lines 4736–4763):

```
4736 if (!auth_decrypt_totp_secret(username, &record, secret) ||
4737     !auth_verify_totp_counter_with_algorithm(secret, record.totp_secret_bytes,
4738                                             totp, record.totp_algorithm, &matched_counter)) {
4740     (void)auth_rate_limit_failure(database, ip_scope);
4742     auth_public_error(error, error_size, "Invalid username, password, or authentication code.");
4743     goto cleanup;
4745 }
4747 if (matched_counter > INT64_MAX || (int64_t)matched_counter <= record.last_totp_counter ||
4748     sqlite3_prepare_v2(database,
4749         "UPDATE users SET last_totp_counter = ?1 "
4750         "WHERE username = ?2 AND last_totp_counter < ?1;", ... ) != SQLITE_OK) {
4753     auth_public_error(error, error_size, "Authentication code was already used.");
4754     goto cleanup;
4755 }
```

6 Authorization, Session Gating, and Abuse Controls

6.1 Role model

The authentication database recognizes three roles: normal user, co-administrator, and primary administrator. Normal authenticated users may use ordinary application operations. Remote administrative operations require an authenticated privileged session. The primary administrator is intentionally restricted to the local server console, while remote privileged sessions are limited to co-administrators.

Relevant code (AuthScreen.c lines 4769–4798):

```
4769 /* Remote sessions may be opened by normal users and co-administrators.
4770    The primary administrator is intentionally restricted to the local server console. */
4776 if (record.role == AUTH_ROLE_ADMIN && !is_local_console) {
4780     auth_public_error(error, error_size,
4781         "The primary administrator may only sign in locally on the server console.");
4782     goto cleanup;
4784 }
4795 *is_admin = record.role == AUTH_ROLE_CO_ADMIN ||
4796     (is_local_console && record.role == AUTH_ROLE_ADMIN);
```

Account-management code separately prevents a co-administrator from modifying the primary administrator. The primary account cannot be deleted, and its role is protected by both application checks and database triggers.

6.2 Pre-authentication and rate limits

Before login, the secure network limits each frame to 206 bytes; after authentication the larger application payload limit applies. The network also limits concurrent clients to 32 and limits TLS handshakes from one address to 20 per minute.

Authentication attempts are rate-limited by stored scopes. Remote authentication uses both an IP scope and a username-plus-IP scope. After five failures within the 15-minute window, lock time starts at 30 seconds and doubles with additional failures up to 900 seconds. Successful authentication clears the corresponding failure state.

The server loop rejects SAVE, LOAD, LIST, DELETE, and user-management operations until authentication succeeds. User-management requests additionally require the privileged session flag.

7 Defensive Memory and File Handling

Security-sensitive buffers are frequently cleared with `OPENSSL_cleanse()` or released with `OPENSSL_clear_free()`. Authentication payloads, passwords, TOTP secrets, exported TLS binding material, ML-DSA proof buffers, and session usernames are explicitly cleared after use.

`SecureFunctions.c` provides bounded copies, checked multiplication/allocation, file-size validation, and private-directory helpers. Security directories must be owned by the effective user and not writable by group/other users; directories are opened with `O_NOFOLLOW` to reject symbolic-link substitution.

Relevant code (`SecureFunctions.c` lines 652–724):

```

652 static bool sec_directory_fd_is_secure(int directory_fd) {
653     if (!S_ISDIR(st.st_mode) || st.st_uid != geteuid())
654         return false;
655     if ((st.st_mode & (S_IWGRP | S_IWOTH)) != 0)
656         return false;
657     return true;
658 }
659
660 directory_fd = open(normalized, O_RDONLY | O_DIRECTORY | O_CLOEXEC | O_NOFOLLOW);
661
662 if (directory_fd < 0 || !sec_directory_fd_is_secure(directory_fd))
663     goto cleanup;

```

8 Cryptographic Primitive Summary

Primitive	Where used	Purpose
ML-DSA-87	Server identity, signed LAN packets, TLS channel proof	Persistent post-quantum digital identity and signature verification.
SHA-512	Public-key fingerprint, identity/key-file digests, server-ID derivation	Domain-separated hashing and integrity/fingerprint construction.
TLS 1.3	SecureNetwork transport	Encrypted and integrity-protected client/server transport.
SecP384r1MLKEM1024	TLS group	Hybrid TLS key establishment selected by OpenSSL.
TLS_AES_256_GCM_SHA384	TLS cipher suite	AES-256-GCM record protection with SHA-384 in the TLS 1.3 suite.
secp384r1 + SHA-384	Ephemeral TLS X.509 certificate	Temporary certificate/key used by the TLS handshake; not the application's long-term trust anchor.
TLS exporter	Channel-bound identity proof	Derives connection-specific binding material that is signed by ML-DSA-87.
Argon2id	Password database	Memory-hard password hashing with $t = 3$, 64 MiB, parallelism 4.
HMAC-SHA512	TOTP and TOTP wrapping-key derivation	Six-digit TOTP generation/verification and two-stage per-user key derivation.
AES-256-GCM	Stored TOTP secret	Confidentiality and authentication for the 32-byte TOTP secret.
RAND_bytes()	Salts, nonces, secrets, TLS serial	Cryptographically secure random values.

Primitive	Where used	Purpose
CRYPTO_memcmp()	Identity-file/key-file digest checks	Constant-time digest comparison.

9 Compact Security Reference

- **Trust anchor:** explicitly imported/selected ML-DSA-87 server public key.
- **Remote channel:** TLS 1.3, SecP384r1MLKEM1024, TLS_AES_256_GCM_SHA384.
- **Server authentication:** ML-DSA-87 signature over TLS exporter binding and fresh client/server nonces.
- **Password storage:** Argon2id, 16-byte salt, 32-byte hash, time cost 3, memory 64 MiB, parallelism 4.
- **TOTP:** 32-byte secret, HMAC-SHA512, six digits, 30-second step, ± 1 verification window, accepted-counter replay protection.
- **TOTP at rest:** AES-256-GCM with 12-byte nonce and 16-byte tag; per-user 256-bit key derived with HMAC-SHA512 from a server master key.
- **Privileges:** user, co-administrator, primary administrator; the primary administrator is local-console only.
- **Abuse controls:** pre-auth payload cap, client cap, handshake-rate limit, login lockouts, authenticated request gating.
- **Local hardening:** bounded copies/allocations, owner-private directories, O_NOFOLLOW, and explicit secret-memory cleansing.